ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Type parametrization and subtyping

Having discussed subtyping and various examples of parametrization at work, it is time for a deeper dive on the interaction betweeen the two mechanisms. In particular, the question we want to address is: given a parametrized data structure `Container[A]`, and any two subtype-related types `S` and `T`, does some relationship between `S` and `T` carry over to `Container[S]` and `Container[T]`?

As we have seen with records in our last lecture, the answer depends on what the datatype does with its components. In the current lecture we discuss how the principles governing record subtyping generalize to parametrized types.

We start by introducing a parametric type that we will use as the running example throughout: a functional `Queue`.

Parametric containers

Queues, like any other container, represent the prototypical example of parametric datatype: a queue has the same shape regardless of what kind of element it holds — `Queue[Int]`, `Queue[String]`, `Queue[Point]` all share the same operations, differing only in the type of the elements.

To keep the focus on parametrization and variance — rather than on data structure design — we adopt a simple, list-based implementation of a functional queue:

```
class Queue[A] private (private val data: List[A]):
  def head: A                 = data.head
  def dequeue: Queue[A]       = new Queue(data.tail)
  def enqueue(a: A): Queue[A] = new Queue(data ++ List(a))

object Queue:
  def apply[A](as: A*): Queue[A] = new Queue(as.toList)
```

The class declaration combines two ideas worth pointing out:

- The keyword `private` *before* the parameter list makes the *primary constructor* private. Clients cannot call `new Queue(...)` directly.
- The parameter `data` is itself declared `private val`, so it becomes a private immutable field of the class — invisible to clients.

Together with the public `apply` factory in the companion object, these declarations produce a proper abstract data type: clients construct queues only through `Queue(1, 2, 3)` (which expands to `Queue.apply(1, 2, 3)`, and from there to `new Queue(List(1, 2, 3))` — a call the companion object is permitted to make), and they manipulate queues only through the exposed methods `head`, `dequeue`, and `enqueue`.

The implementation is purely functional: `enqueue` and `dequeue` produce new queues without modifying any existing ones. We will see shortly that this fact alone does *not* settle the question of variance.

The rest of this lecture is about how subtyping interacts with type constructors: assuming `S <: T`, when (if ever, and why how) does it follow that we may safely take it that `Queue[S] <: Queue[T]`, or `Queue[T] <: Queue[S]`?

Variant parametrized types

With its current definition, our parametric queue type does not support any subtype relationships among its possible instantiations. In fact, Scala parametrized types introduced with *unannotated* type variables, as our `Queue[A]`, are *invariant* in their type parameter: irrespective of the relationship that may hold between two types `S` and `T` instantiating the parameter `A`, `Queue[S]` and `Queue[T]` are simply *different* types.

If we wish to make a parametrized type *variant*, this must be declared explicitly by tagging the type parameter with a *variance annotation* in its definition. Scala provides two such annotations:

- `Queue[+A]` — declares `Queue` as *covariant* in `A`: whenever `S <: T`, then `Queue[S] <: Queue[T]`. The subtype relation flows in the *same* direction as the parameter.
- `Queue[-A]` — declares `Queue` as *contravariant* in `A`: whenever `S <: T`, then `Queue[T] <: Queue[S]`. The subtype relation flows in the *opposite* direction of the parameter.

The choice of invariance as the default is conservative — but, as we shall see, for good reason. Soundness places strong constraints on which variance annotations a type may carry, and not every type that one would *like* to make covariant can be.

Before we explore the annotations, let us first note that invariant containers such as `Queue[A]` do not, per se, rule out useful uses of subtyping. To illustrate, consider the following snippet:

```
val mq: Queue[Shape] = Queue(Circle(1.0), Rectangle(3.5, 4.0))
```

This declaration typechecks even with our invariant `Queue[A]`. The reason is type inference:

- when the compiler sees the expected type `Queue[Shape]` on the left, it applies the queue constructor with `A = Shape`, and
- the arguments `Circle(1.0)` and `Rectangle(3.5, 4.0)` are accepted as values of `Shape`, because `Circle <: Shape` and `Rectangle <: Shape`.

Thus the queue is constructed at the wider element type from the outset, and no subtype relation between `Queue[Circle]` and `Queue[Shape]` or `Queue[Rectangle]` and `Queue[Shape]` is ever needed.

While these uses of subtyping are useful, covarariant definitions would bring additional flexibility.

```
val cq: Queue[Circle] = Queue(Circle(1.0), Circle(2.0))

def use(q: Queue[Shape]): Unit = ...

use(cq)               // needs Queue[Circle] <: Queue[Shape]
```

With invariant queues, this call is rejected. The type of `cq` has already been fixed to `Queue[Circle]`, and the type checker has no way to relate `Queue[Circle]` and `Queue[Shape]`, even though `Circle <: Shape`. Type inference does not come to the rescue: the queue's element type is settled at its declaration (notice that this is true regardless of whether we annotate `cq`: the type inferred would anyway be `Queue[Circle]`), and only a covariant definition of `Queue` would allow the upcast required at the call site.

The subtleties of covariant parametrization

Given that our current implementation of `Queue[A]` is purely functional (has no updates), from the discussion on subtyping one would expect covariant queues to be perfectly safe. As it turns out, the presence of overriding brings subtle, adverse effects.

To see the problem, assume we naively declared our current queue implementation covariant, by tagging the type parameter as `+A`, and consider a subclass that specializes the queue at type `Double`, overriding `enqueue` to do something that depends on the parameter being a `Double`:

```
class PowerQueue extends Queue[Double]:
  override def enqueue(a: Double): Queue[Double] =
    super.enqueue(math.pow(a, 2))        // squares before storing
```

The override is perfectly reasonable on its own: a `PowerQueue` is a `Queue[Double]`, and its `enqueue` method squares its argument before adding it to the queue.

Now, relying on the (covariant) subtype relationship `Queue[Double] <: Queue[Any]`, we could break the system as follow:

```
val q: Queue[Any] = new PowerQueue       // by covariance, Queue[Double] <: Queue[Any]
q.enqueue("abc")                         // typechecks: "abc" is an Any, but crashes at run time
```

Here, the call `q.enqueue("abc")` typechecks because the static type of `q` is `Queue[Any]`, whose `enqueue` method accepts any `Any`. At *runtime*, however, dynamic dispatch sends the call to `PowerQueue.enqueue`, which expects a `Double` and applies `math.pow` to it. Applying `math.pow` to a string fails: the type system has been deceived.

The problem is similar to the one observed with mutable datatypes: when a client calls `enqueue` through the wider type `Any`, dispatchit to the overriding method causes a higher-typed varie to be passed for a lower type parameter, exactly the failure mode we saw above with the `Box` example of mutable records.

A covariant declaration `Queue[+A]` should therefore be rejected, as unsound. Indeed, trying the type checker we get:

```
class Queue[+A] private (private val data: List[A]):
  def enqueue(a: A): Queue[A] = ...

-- Error: ----------------------------------------------------------------------
   |    def enqueue(a: A): Queue[A] = ...
   |                ^^^^
   |covariant type A occurs in contravariant position in type A of parameter a
```

Interestingly, the typing failure is a direct consequence of the structural subtyping rules on datatype and function types.

Remember we said that with inheritance the role of the type system is to validate the `extends` declarations, to make sure that the subclass is indeed a structural subtype of the class it extends. With variance annotation the situation is similar: given the variance annotation `+A`, to accept the definition

the type checker must validate the structural subtype relation `Queue[A] <: Queue[B]`, whenever `A <: B`. That is because, if accepted, the definition would allow all corresponding substitutions.

To see that the validation fails, we may represent the queue type by its public interface — that is, as a record type whose components are the operations the type exposes:

$$\mathsf{Queue}[\mathsf{A}] \;=\; \{\mathsf{head} : \mathsf{A},\ \mathsf{enqueue} : \mathsf{A} \to \mathsf{Queue}[\mathsf{A}],\ \mathsf{dequeue} : \mathsf{Queue}[\mathsf{A}]\}$$

Now, by record subtyping, if we assume $\mathsf{A} <: \mathsf{B}$, proving $\mathsf{Queue}[\mathsf{A}] <: \mathsf{Queue}[\mathsf{B}]$ requires all the component types to vary covariantly. In particular, we would need to show:

$$\mathsf{A} \to \mathsf{Queue}[\mathsf{A}] <: \mathsf{B} \to \mathsf{Queue}[\mathsf{B}]$$

However, this fails because subtyping on function is contravariant on the input type, in this case requiring $\mathsf{B} <: \mathsf{A}$ while we have assumed the opposite ordering.

Checking variance annotations: the position calculus

As an alternative to the verification we just discussed, the Scala type checker relies on a *position calculus* to classify all type parameter *positions* in a class (or trait) as *positive*, *negative*, or *neutral*.

A *position* is any location in the type expressions of the class/trait body where a type parameter may be used:

- every method parameter type is a position,
- every method return type is a position,
- every field type is a position,
- argument and result of a function type is a position.

The classification follows two simple rules:

1. The body of the class is, by default, a positive position.
2. In a function type `S => T`, the sign of input position `S` is *flipped* relative to the surrounding context, while the output position `T` keeps the same sign as the surrounding context.

Once positions are classified:

- A type parameter declared `+A` may only appear in *positive* positions.
- A type parameter declared `-A` may only appear in *negative* positions.
- A type parameter with no annotation may appear *anywhere*.

The two rules compose: if a method has type `(A => B) => C` and we are looking at how `A` appears, we trace through two levels:

- the outer arrow has `(A => B)` in its input (negative) position;
- inside `(A => B)`, `A` is in the input (again negative) position, hence its sign *flips*, and is classified as *positive*

That is the same rationale as the variance behaviour of the input and output position in the subtyping rule for function types: output position as covariant, input positions are contravariant. But nesting a contravariant within a contravariant position makes it covariant: the negative of a negative is positive.

As a worked example, consider the body of class methods of `Queue[A]`:

```
def head: A                          // A in position +
def dequeue: Queue[A]                // A in position +
def enqueue(a: A): Queue[A]          // a:A in position -, return in position +
```

`head` returns an `A`: this is the body's positive position, so `A` appears positively. Same for `dequeue`, which returns a `Queue[A]`. `enqueue`, instead, takes a parameter of type `A`: the parameter type is the input of a function, which flips the sign — so `a: A` puts `A` in a *negative* position.

If `A` is declared `+A`, the negative position is a violation: the type checker reports exactly this, "covariant type `A` occurs in contravariant position." The annotation `+A` says we have promised to use `A` only in positive positions; the method `enqueue` breaks that promise.

The technical mechanisms of the position calculus are subtler than this in some details, but the intuition above captures the essence.

Recovering sound covariant parametrized types

Sound implementations of covariant parametrized types are still possible, but we pay a price in design complexity.

The trick is to replace the occurrences of a covariant type paramenter in contravariant positions with fresh type variables, pushing back the original type variables in covariant positions.

We illustrate the technique with the following revised version of our running example, in which `enqueue` is made polymorphic in a supertype of the type parameter `A`:

```
class Queue[+A] private (private val data: List[A]):
  // ... as before
  def enqueue[U >: A](a: U): Queue[U] = new Queue(data ++ List(a))
```

The new definition of `enqueue` is made parametric / polymorphic in a fresh type variable `U`, but unlike the polymoprphic definitions discussed so far, the new definition uses *bounded polymorphism*: `U` is not intended to range over *all* types, but rather over *all supertypes of* `A`.

We will look at bounded polymophism is greater detail in a separate lecture. Here, what it is important to remark is that, unlike the original definition which requires the types of the new elements to be enqueued to have *exactly* the type `A`, the new version

- accepts arguments of any `U` supertype of `A`, and correspondingly,
- returns a `Queue[U]` instead of `Queue[A]`.

This makes the definition type correct, and sound:

- technically, this is because the the lower bound `U >:A` places `A` in a *positive* position (if `A <: B`, then the set of supertypes of `A` is smaller than the set of supertypes of `B`).
- intuitively, the lower bound `U >:A` anticipates all possible uses of `enqueue` at supertypes of `A` (as enabled by a use of the container queue at wider types) by adjusting the assumption on the type of the enqueued element accordingly.

Indeed, by assuming that the type of the `enqueue` parameter is *any* supertype `A`, we leave the type checker with *no information* on how the parameter can be operated upon (except for being passed around). In particular, a subclass cannot override `enqueue` to specialize on a particular static parameter type — and so the unsoundness witness we exhibited earlier no longer typechecks:

```
class PowerQueue extends Queue[Double]:
  override def enqueue[U >: Double](a: U): Queue[U] =
    super.enqueue(math.pow(a, 2))

-- Type Mismatch Error: ------------------------------------------------
   |    override def enqueue[U >: Double](a: U): Queue[U] =
   |                                               super.enqueue(math.pow(a,2))
   |                                                                      ^
   |           Found:    (a : U)
   |           Required: Double
   |           where:    U is a type in method enqueue with bounds U >: Double
```

The override must accept any `U >: Double`, but its body tries to pass `a` to `math.pow`, which expects a `Double`. The type checker rejects the override at compile time, because there is no guarantee that the runtime argument is a `Double` — it may be any supertype of it. The `PowerQueue` specialization simply cannot exist in the variant world.

Contravariant types and upper bounds

Not surprisingly, there exist datatypes that have a natural contravariant nature. In fact, whereas covariant types are inherently associated with containers that *produce* values, contravariant types arise naturally for datatypes that can be construed as *consumers* of values. Consider, for instance, the case of output channels defined below:

```
trait OutputChannel[-A]:
          def write(x: A) : Unit
```

Though it might at first seem counter-intuitive, the contravariant definition of output channel makes perfect sense. To see that, consider that the only supported operation on a channel of type, say `OutputChannel[String]` is to write a `String` to it. But the same operation can also be done on an `OutputChannel[AnyRef]` (as long as we don't read back from the channel) so it is safe to substitute `OutputChannel[AnyRef]` for `OutputChannel[String]` . As soon as we "read" from the channel, the contravariant annotation is rejected:

```
trait OutputChannel[-A]:
          def write(x: A) : Unit
          def last : A
-- Error: ----------------------------------------------------------------------
   |    def last : A
   |    ^^^^^^^^^^^^
   |contravariant type A occurs in covariant position in type => A of method last
```

The problem is the exact dual of the corresponding problem with covariant definitions, and the solution can be obtained by applying the same duality principle:

```
trait OutputChannel[-A]:
          def write(x: A) : Unit
          def last[U <:A] :  U
```

Supplementary Material — Scala variant types vs Java's wildcards

Variance annotations provide a convenient alternative to traditional mechanisms available in object-oriented languages to enable flexible uses of data structures based on subtyping. One relevant example is Java, with its system of wildcards and (bounded) polymorphic types. We look at the tradeoffs between Java's wildcards and Scala's variant types below.

As we will show, variant types appear superior:

- when designing class code:
  - the subtyping flexibility of variant traits or classes can be achieved with invariant traits and classes,
  - but the price to pay is additional complexity in the design of methods within traits /classes;
- when designing client code:
  - invariant class definitions require effort for clients by means of use-site variance mechanisms.

In both cases, lower bounds and upper bounds play a key role.

We illustrate the point with the design and use of (immutable) `Streams` , and the `filter` and `map` methods which have a related structure in Java and Scala (streams are deprecated in Scala, but the exact same arguments apply to LazyLists). In Scala, we have the following, rather direct definition:

```
trait Stream[+A]:
          . . .
          def filter(p: A => Boolean) : Stream[A]
          def map[B](f: A =>  B): Stream[B]
```

The Java version of the same type is more contrived.

```
interface Stream<T>
    . . .
          Stream<T> filter(Predicate<? super T> p)
          <R> Stream<R> map(Function<? super T,? extends R> f)`
```

To ease the discussion, it is useful to start with reconciling the syntactic conventions in the two languages, by

- "decoding" the types `Predicate<T>` and `Function<T,U>` and exposing their true nature as the function types `T => Boolean` and `T => U`, respectively, and
- expressing the `? super T` and `? extend T` notation in terms of its Scala upper and lower bound correspondents.

Then, we may rewrite the Java definition in Scala syntax as follows:

```
trait JavaStream[T]
    . . .
         def filter[U >:T](p : U => Boolean): JavaStream[T]
         def map[R, U >: T, S <: R] (f : U => S): JavaStream[R] `
```

You may wonder whether the additional structure of lower and upper bounds used in the invariant `JavaStream` trait is really needed: intuitively, that structure allows the two methods to be used at supertypes of the type `T` of the enclosing `JavaStream` so as to recover the flexibility of the variant `Stream[+A]` trait.

To illustrate, we define a function that, given a stream of points and a line, selects the points that belong to the line. The following definitions, using the variant `Stream` trait, serves the purpose:

```
class Line(val m: Double, val q: Double)

def belongs(l : Line)(p: Point): Boolean = (p.y == l.m * p.x + l.q)
def pointsOnLine(pts : Stream[Point], l: Line) = pts.filter(belongs(l))
```

Now assume we wish to reuse the previous definition on a stream of colored points.

With the variant `Stream` trait the reuse is immediate, as `Stream[ColorPoint] <: Stream[Point]` hence a `Stream[ColorPoint]` can be simply passed to `pointsOnLine`.

```
val coloredPoints : Stream[ColorPoint] = . . .
pointsOnLine(coloredPoints, new Line(2,5))
```

With the invariant `JavaStream` instead, we must necessarily refine the definition of our `pointsOnLine` function, so as to make it reusable with streams of subtypes of `Point`. Lower bounds come to the rescue, here, as we can rewrite our function as a polymorphic function with a lower bound:

```
def javaPointsOnLine[U <: Point](pts : JavaStream[U], l: Line) =
          pts.filter(belongs(l))

val javaColoredPoints : JavaStream[ColorPoint] = . . .
javaPointsOnLine(javaColoredPoints, new Line(3,5))
```

The type checker accepts `javaPointsOnLine(javaColoredPoints)` by instantiating `U` to `ColoredPoint`, which is correct as `ColoredPoint <: Point` : that is how the combined use of a polymorphic stream and subtyping of the component type captures the effect of subtyping over the stream type.

It is instructive to look at the invocation `pts.filter(belongs(l))` and to detail its typing, as it this is where the polymorphic type bound used in the definition of `JavaStream.filter` comes into the picture. In fact.

- on the one hand `pts : JavaStream[U]` with `U <: Point`
- on the other `filter` accepts predicates whose input type is a supertype of `U` :
- but that's exactly the case for `belongs(l): Point => Boolean` , given that `U <: Point`

Viewed differently, `belongs(l) : Point => Boolean` may legally be passed for the the parameter `p: U => Boolean` , because `Point => Boolean <: U => Boolean` given that `U <: Point` (and the fact that function types are contravariant in their input).

References and Credits

The first part of this lecture is based on Chapter 18 of *Programming in Scala*.

The comparison between Scala and Java is inspired by (but substantially different from) S. Crafa's lecture note and slides from her course on Advanced Topics in Programming Languages

Upper bounds are an application of the principle of *bounded quantification* introduced by L. Cardelli and P. Wegner in their renown paper on Understanding Types, Data Abstraction and Polymorphism.